

User Manual

Football Tournament Management

System

Rostand Cedrix Assonfo Demaboi

Summer Term 2026

Technische Hochschule Georg Agricola, Bochum

Module: Introduction to Database Management Systems

Dozent: Stephan Bökelmann

September 12, 2026

Contents

1	Introduction and System Overview	1
1.1	Organizer Mode	1
1.2	Viewer Mode	1
2	Backend Deployment and Launch	1
2.1	Prerequisites	1
2.2	Deployment Using Docker Compose	2
2.3	Separate Backend and Frontend Launch	2
2.3.1	Terminal 1 – Start the Backend	2
2.3.2	Terminal 2 – Start the Frontend	3
2.4	Difference Between the Two Launch Methods	3
3	Frontend Launch and Connection	3
3.1	Activating the Python Environment	3
3.2	Startup Connection Dialog	4
3.3	Viewer Mode	4
3.4	Organizer Mode	4
4	Interface Navigation and Usage	4
4.1	Tournaments Tab	4
4.2	Teams Tab	5
4.3	Matches Tab	5
4.4	Standings Tab	5
5	Administrative Operations	6
5.1	Scheduling a Match	6
5.2	Updating a Match Score	6
5.3	Deleting Data	7
5.4	Refreshing the Data	7
5.5	Logout and Switching User Mode	7
6	Typical Usage Workflow	7
7	Troubleshooting	8
7.1	Docker Containers Are Not Running	8
7.2	Frontend Cannot Connect to the Backend	8
7.3	Backend Started Separately	8
7.4	Virtual Environment Cannot Be Activated	8
8	Conclusion	9

1 Introduction and System Overview

The Football Tournament Management System is an application designed to organize, track, and administer football competitions.

The system follows a decoupled architecture consisting of three main components:

- **Backend:** A REST API developed with FastAPI and connected to a relational database.
- **Frontend:** A desktop graphical interface developed with Python Tkinter for user interaction.
- **Orchestration:** Docker Compose is used to simplify deployment and ensure a reproducible runtime environment.

The application provides two main access modes.

1.1 Organizer Mode

Organizer Mode provides access to administrative operations.

Authenticated organizers can create, modify, and delete tournaments, teams, and matches.

Authentication is performed using an API token.

1.2 Viewer Mode

Viewer Mode provides read-only access to the application.

Users can view tournaments, teams, matches, scores, and rankings without modifying the database.

2 Backend Deployment and Launch

The backend server and database can be started using Docker Compose or the backend can be launched separately for development and testing.

This section describes both methods.

2.1 Prerequisites

Before starting the application, make sure that the following software is installed:

- Docker
- Docker Compose
- Python 3

2.2 Deployment Using Docker Compose

Open a terminal and navigate to the project root directory:

```
cd ~/football-tournament-manager
```

Start the backend and database containers:

```
docker compose up --build -d
```

The `-build` option ensures that the required Docker images are rebuilt when necessary, while `-d` starts the services in detached mode.

Check the status of the containers with:

```
docker compose ps
```

The containers should be displayed as running.

The FastAPI backend is available on port:

```
8000
```

The default local API URL is:

```
http://localhost:8000
```

2.3 Separate Backend and Frontend Launch

The application can also be started without using Docker Compose for the frontend.

In this configuration, the backend is started in one terminal and the frontend is started in a second terminal.

This method is useful for development and testing because the frontend and backend can be started and monitored independently.

2.3.1 Terminal 1 – Start the Backend

Open the first terminal and navigate to the backend directory:

```
cd ~/football-tournament-manager/backend
```

Activate the Python virtual environment if one is configured for the backend:

```
source .venv/bin/activate
```

Start the FastAPI backend with:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Keep this terminal open while using the application.

The backend should then be available at:

```
http://localhost:8000
```

2.3.2 Terminal 2 – Start the Frontend

Open a second terminal.

Navigate to the frontend directory:

```
cd ~/football-tournament-manager/frontend
```

Activate the frontend virtual environment:

```
source .venv/bin/activate
```

Start the graphical application:

```
python3 src/football_tournament/main.py
```

The frontend connects to the backend running in the first terminal.

In the connection dialog, use:

```
http://localhost:8000
```

For Organizer Mode, enter:

```
mykey
```

For Viewer Mode, leave the API token empty or enter an incorrect token.

2.4 Difference Between the Two Launch Methods

There are two possible ways to run the application.

With Docker Compose:

```
docker compose up --build -d
```

Docker manages the backend and database services.

With separate terminals, the backend and frontend are started independently.

The separate method is particularly useful during development because changes to the Python backend or frontend can be tested directly.

It is important to ensure that both launch methods use the same database configuration if identical initial data is expected.

3 Frontend Launch and Connection

Once the backend is running, the desktop frontend can be started.

3.1 Activating the Python Environment

Navigate to the frontend directory:

```
cd ~/football-tournament-manager/frontend
```

The project uses a virtual environment named `.venv`. Activate it with:

```
source .venv/bin/activate
```

Then start the application with:

```
python3 src/football_tournament/main.py
```

3.2 Startup Connection Dialog

When the application starts, a connection dialog is displayed.

The dialog requests the following information:

- **API URL:** The address of the FastAPI backend. The default value is `http://localhost:8000`.
- **X-API-Token:** The authentication token used to access organizer operations.

3.3 Viewer Mode

To access the application in read-only mode, leave the **X-API-Token** field empty or enter an incorrect API token.

In both cases, the application starts in Viewer Mode.

3.4 Organizer Mode

To access administrative functions, enter the API token configured for the application.

The API token used by the application is:

mykey

Only the correct token allows access to Organizer Mode.

If the token is empty or incorrect, the application connects in Viewer Mode.

4 Interface Navigation and Usage

The main interface is organized as a tabbed dashboard.

The available sections allow users to inspect and manage the main entities of the football tournament system.

4.1 Tournaments Tab

The Tournaments tab displays the registered football competitions.

Users can view information such as:

- Tournament name
- Start date
- End date
- Current status

Organizers can create, modify, or delete tournaments when authenticated with a valid API token.

4.2 Teams Tab

The Teams tab displays the participating teams.

For each team, the interface provides information such as:

- Unique team identifier
- Team name
- Coach or manager name

Organizers can create, update, and delete teams.

The current application interface does not provide a dedicated player-management workflow in this section. Therefore, players should not be considered a directly managed feature of the current graphical interface.

4.3 Matches Tab

The Matches tab displays the fixtures registered in the system.

A tournament selector at the top of the tab allows users to filter the displayed matches.

The available options include:

- A specific tournament
- All tournaments

The match table displays information such as:

- Match date
- Home team
- Away team
- Score
- Referee

For a match that has not yet received a score, the score is displayed using a dash (-). Validated scores are displayed using the Home–Away format.

4.4 Standings Tab

The Standings tab displays the current ranking of teams in a tournament.

The ranking is calculated dynamically from the recorded match results.

The displayed statistics include:

- Points
- Wins
- Draws

- Losses
- Goals scored
- Goals conceded
- Goal difference

The standings therefore reflect the current results stored in the database.

5 Administrative Operations

Administrative operations are available to users authenticated in Organizer Mode.

The following operations can be performed from the graphical interface.

5.1 Scheduling a Match

To create a new match:

1. Open the **Matches** tab.
2. Click the **+ Schedule Match** button.
3. Select the tournament.
4. Select the home team.
5. Select the away team.
6. Select or enter the match date.
7. Enter the referee information if required.
8. Confirm the operation using the **SAVE MATCH** button.

The new match is then stored in the database and becomes available in the Matches tab.

5.2 Updating a Match Score

To update the score of an existing match:

1. Open the **Matches** tab.
2. Select the desired match.
3. Click **Update Score**.
4. Enter the goals scored by the home team.
5. Enter the goals scored by the away team.
6. Confirm the modification.

The new result is stored in the database.

The standings are subsequently recalculated from the updated match results.

5.3 Deleting Data

Organizers can delete obsolete teams or matches using the corresponding **Delete** operation.

Deletion is subject to the database's referential integrity constraints.

If an object is referenced by another database object, the database may prevent the deletion in order to preserve data consistency.

5.4 Refreshing the Data

The application provides a **Refresh Data** function.

This operation retrieves the current data from the backend API and updates the local graphical display.

It is particularly useful when data has been modified externally or by another client.

5.5 Logout and Switching User Mode

To terminate the current session, close the application or use the available disconnect/logout functionality in the connection panel.

When the application is started again, the connection dialog is displayed and the user can choose whether to connect in Viewer Mode or Organizer Mode.

6 Typical Usage Workflow

A typical tournament-management workflow can be performed in the following order:

1. Start the backend and database using Docker Compose, or start the backend separately in the first terminal.
2. Verify that the backend is running.
3. Open a second terminal and start the frontend application.
4. Enter the backend API URL.
5. Enter the API token to access Organizer Mode, or leave it empty for Viewer Mode.
6. Create a tournament.
7. Create the participating teams.
8. Schedule the matches between the teams.
9. Enter the results after matches have been played.
10. Open the Standings tab to view the updated ranking.
11. Use Refresh Data whenever information needs to be synchronized with the backend.

This workflow allows an organizer to manage a complete football competition from its creation through the recording of match results and rankings.

7 Troubleshooting

7.1 Docker Containers Are Not Running

If the containers are not running, execute:

```
docker compose ps
```

To inspect the container logs, use:

```
docker compose logs
```

The backend can also be restarted with:

```
docker compose up --build -d
```

7.2 Frontend Cannot Connect to the Backend

First verify that the backend is running.

The default API address is:

```
http://localhost:8000
```

Also verify that the API token is correct when using Organizer Mode.

7.3 Backend Started Separately

If the backend is started manually, open the first terminal and execute:

```
cd ~/football-tournament-manager/backend
```

Then start the FastAPI application:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Keep this terminal running.

Open a second terminal and execute:

```
cd ~/football-tournament-manager/frontend
source .venv/bin/activate
python3 src/football_tournament/main.py
```

The frontend must connect to:

```
http://localhost:8000
```

7.4 Virtual Environment Cannot Be Activated

Make sure that you are inside the frontend directory:

```
cd ~/football-tournament-manager/frontend
```

Then activate the correct environment:

```
source .venv/bin/activate
```

If the virtual environment does not exist, it must be created according to the project's Python environment setup instructions.

8 Conclusion

The Football Tournament Management System provides a complete interface for managing football tournaments, teams, matches, scores, and standings.

The system separates the graphical frontend from the REST API and database layer. Docker Compose provides a reproducible environment for running the backend and database.

The application can also be launched separately, with the backend running in one terminal and the frontend running in a second terminal.

Viewer Mode allows participants to inspect tournament information, while Organizer Mode provides the administrative operations required to maintain the competition.

For development and deployment, the most important components are the Docker Compose environment, the FastAPI backend, the Python virtual environment, and the Tkinter frontend.